

My first step was not to write code. I reread the scenario several times to understand the actors, information flows, and likely points of failure. I paid special attention to words like “may”, “usually”, “sometimes”, and “rarely”, because those are where operational edge cases tend to hide. I eventually concluded that the problem was not simply “send weekly food orders.” The deeper issue was that too much memory, judgement, and repetitive coordination was concentrated in the program coordinator.

I also tried to apply Elon Musk’s “delete first” principle. I asked whether catering should exist at all. Removing it probably would not collapse the business, but I decided it was still worth preserving. Catering reduces pressure on parents, differentiates Padea, gives students energy and a social break, and acts as a small reward for showing up and working hard. But those benefits only exist if students receive food they can eat and enjoy. A meal that violates an allergy, or that a student hates, is not a reward. This became a core requirement: improve meal suitability rather than relying on the coordinator’s “educated guess.”

The task sheet says the coordinator chooses a few items and guesses quantities. That felt like the key bottleneck. The work scales with the number of sessions, and student preference knowledge either lives in someone’s memory, scattered feedback notes, or nowhere at all. My first design decision was therefore to collect each student’s preferred meal and dietary requirements during onboarding. Dylan confirmed that onboarding already happens through a form, so adding food fields is a low-friction improvement. Instead of guessing, the system can query known preferences.

I deliberately tried to minimise friction. Students should not have to constantly fill out forms, and managers should not need to handle every edge case manually. For that reason, I chose to collect preferences only for the student’s primary caterer upfront. Collecting preferences for every possible backup caterer seemed excessive. If a backup caterer is needed, the system can reach out with a one-off SMS/email form. This pushes friction into the rare case instead of the normal case.

The prototype became a Supabase-backed operations system. I modelled sessions, students, student-session membership, caterers, menus, food profiles, absences, exclusions, managers, caterer contacts, orders, order lines, caterer-school capacity, and exceptions. Python handles deterministic order generation; Streamlit provides a lightweight coordinator dashboard. The generator reads database state, applies absences and exclusions, resolves attendees, selects preferred or backup meals, checks dietary compatibility, aggregates quantities, creates caterer email drafts, and produces manager distribution sheets.

A key decision was to keep the core ordering logic deterministic and auditable. I did not want an LLM deciding what a student with an allergy should eat. LLMs make more sense at the messy boundary: parsing family emails, manager notes, or ambiguous preference changes into structured proposed updates. The database remains the source of truth, and safety-critical changes can still be reviewed.

The biggest learning was that complexity hides in the final 1%. The simple form-based idea handles most cases, but edge cases quickly multiply: absences, mid-semester changes, full cancellations, partial year-level exclusions, missing contacts, caterer minimums, backup caterers, and confirmation failures. I built an exception system so the software does not silently pretend everything is fine. Missing mobile numbers, unsafe meal assignments, below-minimum orders, and overdue confirmations become visible dashboard items.

I also modelled the order lifecycle: ready for review, approved, sent to caterer, confirmed, overdue, backup required, and delivered. I separated “confirmation overdue” from “backup required” because overdue is a fact, while backup is an operational decision. A late caterer reply might still be accepted.

Not everything went smoothly. Some schema choices that seemed elegant created refactoring pain. Dropping duplicated fields like `orders.school_id` improved normalisation but broke assumptions in the code. The useful distinction I arrived at was: duplicated truth is usually bad, but decision snapshots can be valuable. For example, a future `order_student_map` table may be worth storing because it records which student was assigned which meal at the time the order was generated.

The prototype is not production-ready. It does not send real emails or SMS messages, and family email handling is simulated. Absence and preference updates currently demonstrate changed state by regenerating orders; production should regenerate only the affected draft order or create an exception if the order has already been sent. Still, the prototype proves the operating loop: database state in, draft orders and exceptions out, coordinator review, caterer confirmation tracking, and family updates flowing back into future orders.

The main lesson was that operations engineering is less about writing a clever script and more about making hidden state explicit. Once preferences, absences, exclusions, contacts, menus, and confirmation status are modelled clearly, much of the coordinator’s repetitive work can be automated. The human role shifts from remembering and guessing to reviewing exceptions and making judgement calls.